CC1101Radio User Manual

ESP-IDF driver for the TI CC1101 sub-GHz transceiver. Component version 1.0.0, requires ESP-IDF 5.0 or newer.

1. Introduction

CC1101Radio is a C++ driver component for the Texas Instruments CC1101 low-power sub-GHz transceiver. It is written against the native ESP-IDF `spi_master` driver (there is no Arduino layer). It was reviewed against TI datasheet SWRS061I, and comments in the source refer to that document's section and table numbers.

The driver has two layers:

- **High-level helpers:** `begin()`, `setFrequency()`, `setBitrate()`, `startReceive()`, `transmit()`, `readPacket()` and so on. These cover the common packet-radio use cases.
- **Raw access:** `readReg()`, `writeReg()`, `strobe()`, `readBurst()` and `writeBurst()`, plus the complete register map in the `CC1101Reg` namespace, for anything the helpers do not cover.

Key facts at a glance:

- Supported bands: 300-348 MHz, 387-464 MHz and 779-928 MHz. The CC1101 cannot do 2.4 GHz.
- Modulations: 2-FSK, GFSK, 4-FSK, MSK and ASK/OOK.
- Several radios can share one SPI bus. Create one `CC1101Radio` object per chip, each with its own CS pin.
- Packet modes: variable length (default), fixed length and infinite (streaming). Also hardware address filtering, CRC, data whitening, Manchester encoding and convolutional FEC.
- Operation is polled. The driver does not use the chip's GDO pins or any interrupts.
- A 26 MHz crystal is assumed (standard on CC1101 modules). All frequency, baud rate, deviation and bandwidth calculations use 26 MHz.

2. Hardware requirements and wiring

2.1 Connections

Signal	Connect to	Notes
SCK	The bus `sclk_io_num` pin	SPI mode 0
MOSI (SI)	The bus `mosi_io_num` pin	
MISO (SO)	The bus `miso_io_num` pin	Also pass this same GPIO to the constructor as `misoPin` (see 2.2)
CSn	Any output-capable GPIO	Driven by the driver as a plain GPIO, not by the SPI peripheral
VCC / GND	3.3 V supply	The CC1101 is a low-voltage part. Never connect it to 5 V
GDO0 / GDO2	Optional	Not used by the driver. GDO0 is configured to assert on sync word and deassert at end of packet (IOCFG0 = 0x06), so you may route it to a GPIO for your own use. GDO2 stays at the chip's reset default

The pin numbers in the example `main.cpp` (SCK 18, MISO 19, MOSI 23, CS 5) are placeholders. Change them to match your wiring.

2.2 The MISO pin and the chip-ready wait

The CC1101 signals that its crystal is running and it is ready for a command by driving SO (MISO) low after CS goes low. The driver can wait for this signal, but only if it knows which GPIO MISO is on. That is the purpose of the `misoPin` constructor argument.

- Pass the same GPIO you gave the bus as `miso_io_num`. Before every register access the driver then waits (up to 2 ms) for MISO to go low.
- Pass `-1` to skip the wait. After reset and after wake-up the driver then falls back to a fixed delay of up to 1 ms instead of watching the pin.

With several radios on one bus, give every radio the same `misoPin`.

2.3 SPI clock speed

The default SPI clock is 4 MHz. Whatever you pass as `spiHz` is clamped to 6.5 MHz, the datasheet limit for burst access (Table 22). The driver uses SPI mode 0.

2.4 Crystal, antenna and matching

- The driver assumes a 26 MHz crystal. A module with a different crystal will transmit and receive on the wrong frequency and at the wrong baud rate, because every calculation depends on it.
- The antenna and matching network must suit your band (315, 433, 868 or 915 MHz). Output power steps are nominal datasheet values. Real output depends on your board.
- On 868/915 MHz boards, the recommended output power settings depend on whether the matching network uses multilayer or wire-wound inductors. See `setInductorType()`.

3. Installing the component

3.1 File layout

The component's `CMakeLists.txt` declares `INCLUDE_DIRS "include"`, so the header must live in an `include` subfolder. The files you downloaded are flat, so arrange them like this:

```
your_project/
  CMakeLists.txt
  components/
    CC1101Radio/
      CMakeLists.txt
      idf_component.yml
      CC1101Radio.cpp
      include/
        CC1101Radio.h
  main/
    CMakeLists.txt
    main.cpp
```

In `main/CMakeLists.txt`, register main with the component as a requirement so the header is found:

```
idf_component_register(SRCS "main.cpp"
                       INCLUDE_DIRS "."
                       REQUIRES CC1101Radio driver esp_timer freertos)
```

3.2 What the component files contain

`CMakeLists.txt` registers one source file, `CC1101Radio.cpp`, with include directory `include`. It requires the ESP-IDF components `driver`, `esp_timer`, `esp_rom` and `freertos`.

`idf_component.yml` is the component manager manifest. It gives the component version `1.0.0`, a description, and one dependency: ESP-IDF 5.0 or newer.

3.3 Language note

The driver is C++ only. If you call it from `app_main`, define `app_main` with `extern "C"` as the example does.

3.4 Building

Build, flash and monitor as usual:

```
idf.py build flash monitor
```

4. Core concepts

4.1 Baud rate versus data rate

The bitrate you set is the **baud (symbol) rate**, not necessarily the data rate. With Manchester encoding on, the data rate in kbps is half the baud rate. With 4-FSK it is twice the baud rate. With FEC on, it is half. `bitrate()` always reports the baud rate.

4.2 Configuration setters drop the chip to IDLE

The chip requires that packet handling, frequency and modem fields only be changed in IDLE. So almost every configuration setter calls `idle()` first. After configuring, call `startReceive()` to go back to receiving. The exception is the output power functions, which do not change state (see 6.6).

4.3 The receive cycle

The driver configures the chip (MCSM1 = 0x00) to go to IDLE after a packet has been received and after a packet has been transmitted. This has three practical consequences:

1. After `transmit()` the radio is idle. Call `startReceive()` if you want to listen again. The example does this.
2. After the chip finishes receiving a packet it sits in IDLE until you call `readPacket()`, which reads the packet out and restarts reception itself.
3. A second packet that arrives while the chip is idle (before your next `readPacket()` call) is missed. Poll often enough for your traffic.

4.4 What goes on air

Variable length mode (default):

```
[preamble][sync word][length][address (optional)][payload][CRC16]
```

Fixed length mode has no length byte:

```
[preamble][sync word][address (optional)][payload][CRC16]
```

Infinite mode has no length byte and no CRC. After the sync word the receiver keeps streaming bytes until you call `idle()`.

The preamble and sync word are produced and detected by the chip. The CRC is 16 bits and is appended and checked by the chip when CRC is enabled. When the chip receives a packet, it appends two status bytes (RSSI, then LQI with the CRC-OK flag in bit 7) to the RX FIFO behind the payload. The driver reads these for you and reports them in `PacketInfo`.

4.5 What must match on both ends

For two radios to talk, both must agree on:

- Carrier frequency, channel number and channel spacing
- Modulation and baud rate, plus deviation for FSK modes
- Sync word and sync mode
- Length mode (and fixed length, if used)
- CRC on or off
- Whitening on or off

- Manchester on or off
- FEC on or off
- Address filtering convention, if used

The receiver's bandwidth also has to be wide enough for the signal. As a rough rule of thumb (not from the driver), RX bandwidth should be at least baud rate plus twice the deviation. The default 4.8 kBd, 47.6 kHz deviation and 135 kHz bandwidth satisfy this.

4.6 Return value conventions

Functions that configure the radio return `bool`: `true` on success and `false` if the argument is out of range or unsupported. When a setter returns `false`, it has not changed the radio. Functions that talk to a packet return `int16_t`, where negative values are errors (section 8.1) and zero or positive values carry a result.

4.7 Threading

Each SPI transaction acquires the SPI bus for its duration, so several radios (or other devices) can safely share the bus from different tasks. The `CC1101Radio` object itself has no locking, and its calls consist of multiple SPI transactions. Use each radio object from one task, or protect it with your own mutex.

5. Quick start

The supplied `main.cpp` is a complete example. It initialises one radio on SPI2, receives continuously, and sends the text `hello` every 5 seconds. Here is what each part does.

Step 1: initialise the SPI bus. The driver does not initialise the bus. You must call `spi_bus_initialize()` first:

```
spi_bus_config_t bus = {};
bus.sclk_io_num = PIN_SCK;
bus.miso_io_num = PIN_MISO;
bus.mosi_io_num = PIN_MOSI;
bus.quadwp_io_num = -1;
bus.quadhd_io_num = -1;
bus.max_transfer_sz = 64;
ESP_ERROR_CHECK(spi_bus_initialize(SPI2_HOST, &bus, SPI_DMA_DISABLED));
```

The driver splits transfers into pieces of at most 64 bytes and sends them with polling transactions. This is the configuration the example is built around (DMA disabled, 64-byte transfer size).

Step 2: create the radio object. Give it the SPI host, the CS pin and the MISO pin. Declare it `static` (or global) so it outlives the function:

```
static CC1101Radio radio(SPI2_HOST, PIN_CS, PIN_MISO);
```

Step 3: call `begin()`.

```
int16_t rc = radio.begin(433.92f, 4.8f, 47.6f, 135.0f,
CC1101Radio::MOD_GFSK);
if (rc != CC1101Radio::ERR_NONE) { /* handle error */ return; }
```

The arguments are frequency in MHz, baud rate in kBd, FSK deviation in kHz, RX bandwidth in kHz and modulation. These are also the defaults, so `radio.begin()` with no arguments does the same thing. After `begin()` the chip is in IDLE with 0 dBm output power.

Step 4: optionally check the chip identity. `radio.partNumber()` and `radio.version()` read the chip's identification registers. The example prints them.

Step 5: start receiving. `radio.startReceive()` flushes the RX FIFO and puts the chip in RX.

Step 6: poll for packets. In your loop, call `readPacket()`. A result greater than zero is the payload length. Zero means nothing complete yet. A negative value is an error. `ERR_CRC` simply means the packet failed its CRC check (the example ignores it quietly):

```
uint8_t buf[CC1101Radio::MAX_PAYLOAD];
CC1101Radio::PacketInfo info;
int16_t n = radio.readPacket(buf, sizeof buf, &info);
```

Step 7: transmit. `radio.transmit(data, len)` blocks until the packet is sent and returns `ERR_NONE` on success. Because the chip is idle afterwards, call `radio.startReceive()` straight after to resume listening.

Step 8: yield. The example ends each loop pass with `vTaskDelay(pdMS_TO_TICKS(5))`, which also sets the polling interval.

6. API reference

6.1 Types and constants

All of these are members of `class CC1101Radio`.

`Modulation` (the value is the chip's MDMCFG2.MOD_FORMAT field):

Name	Value	Meaning
`MOD_2FSK`	0	2-FSK
`MOD_GFSK`	1	Gaussian-filtered FSK (the default)
`MOD_ASK_OOK`	3	ASK / OOK
`MOD_4FSK`	4	4-level FSK
`MOD_MSK`	7	Minimum-shift keying

`SyncMode` (MDMCFG2.SYNC_MODE, datasheet Table 30). The `_CS` variants additionally require carrier sense above threshold:

Name	Value	Meaning
`SYNC_NONE`	0	No preamble or sync is sent in TX and none is detected in RX
`SYNC_15_16`	1	15 of 16 sync bits must match
`SYNC_16_16`	2	16 of 16 must match (the default)
`SYNC_30_32`	3	32-bit sync (SYNC1:SYNC0 sent twice), 30 of 32 bits must match
`SYNC_CS_ONLY`	4	No preamble or sync, carrier sense above threshold
`SYNC_15_16_CS`	5	15 of 16 plus carrier sense
`SYNC_16_16_CS`	6	16 of 16 plus carrier sense
`SYNC_30_32_CS`	7	30 of 32 plus carrier sense

`AddressFilter` (PKTCTRL1.ADR_CHK, datasheet 15.3.1):

Name	Value	Meaning
`ADDR_OFF`	0	No address check
`ADDR_NODE`	1	Accept only packets addressed to our node address
`ADDR_NODE_BC0`	2	Node address plus 0x00 broadcast
`ADDR_NODE_BC0_BCFF`	3	Node address plus 0x00 and 0xFF broadcast

`LengthMode` (PKTCTRL0.LENGTH_CONFIG):

Name	Value	Meaning
`LEN_FIXED`	0	No length byte on air. Both ends agree on the size
`LEN_VARIABLE`	1	A length byte is sent first (the default)
`LEN_INFINITE`	2	No length at all. Bytes stream through the FIFO

`InductorType`: `INDUCTOR_MULTILAYER` (0, the default) or `INDUCTOR_WIREWOUND` (1). Selects which PATABLE set is used at 868/915 MHz (datasheet Tables 37 and 39). Other bands are unaffected.

`MAX_PAYLOAD`: 61. The FIFO is 64 bytes, minus the length byte and the two appended status bytes. With address filtering, the address byte counts as part of these 61 bytes.

`PacketInfo`: filled in by `readPacket()`:

Field	Type	Meaning
`rssi`	`float`	Signal strength of the packet in dBm
`lqi`	`uint8_t`	Link quality indicator (7-bit value from the chip)
`crcOk`	`bool`	The chip's CRC-OK flag for this packet
`hasAddress`	`bool`	True when address filtering is on, meaning `buf[0]` is the destination address
`address`	`uint8_t`	Destination address the packet was sent to (valid only if `hasAddress`)

Field	Type	Meaning
`broadcast`	`bool`	True when the packet matched a broadcast address rather than ours

Error codes are listed in section 8.1.

6.2 Construction, initialisation and reset

Constructor

```
CC1101Radio(spi_host_device_t host, int csPin, int misoPin = -1, uint32_t
spiHz = 4000000);
```

- `host`: the SPI host you passed to `spi_bus_initialize()`, for example `SPI2_HOST`.
- `csPin`: the GPIO used as chip select. The class drives it itself.
- `misoPin`: the GPIO of the bus MISO line, for the chip-ready wait (section 2.2). `-1` skips the wait.
- `spiHz`: SPI clock in Hz. Clamped to 6.5 MHz.

The constructor only stores these values. It does not touch the hardware. The object cannot be copied. The destructor removes the device from the SPI bus.

`int16_t begin(freqMHz = 433.92f, bitrateKbps = 4.8f, devKHz = 47.6f, rxBwKHz = 135.0f, mod = MOD_GFSK)`

Call `spi_bus_initialize()` first. `begin()` then does the following, in order:

1. Adds the device to the SPI bus and configures the CS pin as an output driven high. Fails with `ERR_SPI` if this does not succeed (for example the bus was not initialised or no device slot is free).
2. Resets the chip (see `reset()` below).
3. Reads the VERSION register. A value of 0x00 or 0xFF means no chip answered, and `begin()` returns `ERR_CHIP_NOT_FOUND`. Genuine parts report 0x04 or 0x14, but any other value is accepted.
4. Resets the driver's internal record of the chip state to match the freshly reset chip.
5. Writes a set of sane register defaults (table in section 8.2), based on TI SmartRF Studio output for 433 MHz GFSK at 4.8 kBd.

6. Applies your settings: modulation, then baud rate (checked against that modulation), then deviation, RX bandwidth and frequency. Output power is set to 0 dBm.

Details worth knowing:

- Deviation is only applied for 2-FSK, GFSK and 4-FSK. For ASK/OOK it is ignored. For MSK the argument is ignored and the phase-transition field is written as 0 (use `setMskPhaseTransition()` to change it).
- If any setting is rejected, `begin()` returns `ERR_BAD_ARG`. The chip is then left partly configured, so correct the argument and call `begin()` again.
- `begin()` can safely be called again to fully re-initialise the radio.
- On success the chip is in IDLE, not RX. Call `startReceive()` to listen.

`void reset()`

Performs the datasheet's manual reset sequence (section 19.1.2, Figure 27): CS toggling, the SRES command with waits for the chip-ready signal, and a short settling delay. It also clears the driver's record that the chip is asleep.

`reset()` returns every chip register to its power-on default but does **not** reapply your configuration. The driver's internal record of your settings becomes stale. To reset and reconfigure, call `begin()` again instead. Only call `reset()` after `begin()` has attached the device.

6.3 Radio configuration

Every function in this section drops the chip to IDLE (when it accepts the argument). Call `startReceive()` afterwards.

`bool setFrequency(float mhz)`

Sets the base carrier frequency. Accepted ranges: 300-348, 387-464 and 779-928 MHz. Anything else returns `false` and changes nothing. The resolution is about 397 Hz (the frequency word is `f * 65536 / 26 MHz`, rounded).

If the new frequency falls in a band that uses a different recommended PA table (below 360 MHz uses the 315 MHz table, 360 to below 779 the 433 MHz table, 779 to below 900 the 868 MHz table, 900 and above the 915 MHz table), the output power is re-selected for the new band, unless you set a raw PA value with `setPaRaw()`.

The actual on-air frequency is the base frequency plus the channel number times the channel spacing (see `setChannel()`). `frequency()` returns the base frequency you last set.

The chip auto-calibrates its synthesiser on each IDLE to RX/TX transition (MCSM0 = 0x18), so no manual calibration is normally needed after a frequency change.

`bool setBitrate(float kbps)`

Sets the baud rate in kBd. Valid ranges depend on the modulation (datasheet Table 3):

Modulation	Allowed baud rate
2-FSK	0.6 to 500 kBd
GFSK	0.6 to 250 kBd
ASK/OOK	0.6 to 250 kBd
4-FSK	0.6 to 300 kBd
MSK	26 to 500 kBd

Outside the range, it returns `false` and changes nothing. The chip can only produce certain rates (an exponent and an 8-bit mantissa), so the driver picks the closest one. `bitrate()` returns the rate actually programmed. For example, a request of 4.8 gives about 4.798 kBd.

Remember that Manchester, FEC and 4-FSK change the data rate relative to the baud rate (section 4.1).

`float bitrate() const` and `float frequency() const`

Return the actual programmed baud rate in kBd, and the base frequency in MHz.

`static bool bitrateSupported(Modulation m, float kbps)`

Returns `true` if `kbps` is within the allowed range for modulation `m` (table above). Use it to check a combination before applying it.

`bool setDeviation(float kHz)`

Sets the FSK frequency deviation. Accepted range is 1.5 to 381 kHz, otherwise `false`. The driver searches the chip's exponent/mantissa combinations for the closest achievable value. It is meant for 2-FSK, GFSK and 4-FSK, and is not checked against the current modulation. Do not use it in MSK mode, where the same register holds the phase-transition setting (use `setMskPhaseTransition()` there). It has no effect in ASK/OOK.

`bool setRxBandwidth(float kHz)`

Sets the receiver channel filter bandwidth. Accepted range is 58 to 812 kHz, otherwise `false`. The chip offers 16 discrete bandwidths (section 8.4), and the driver picks the closest.

It also applies TI application note DN022 receive settings that depend on bandwidth: for 325 kHz and below, FIFOTHR ADC retention on, TEST2 = 0x81, TEST1 = 0x35; above 325 kHz, ADC retention off, TEST2 = 0x88, TEST1 = 0x31. In ASK/OOK mode it also sets FREND1 to 0x56 for 102 kHz and below, or 0xB6 above.

`bool setModulation(Modulation m)`

Switches modulation. Returns `false` only for an invalid value. Behaviour:

- It does **not** check your current baud rate. Verify with `bitrateSupported()` and re-call `setBitrate()` if needed.
- Switching to MSK or 4-FSK automatically turns Manchester off (they cannot be combined).
- Switching to ASK/OOK disables frequency offset compensation (FOCCFG), applies OOK AGC settings (AGCCTRL2/1/0 = 0x04/0x00/0x92, and FREND1 chosen from the RX bandwidth) and selects the OOK PA arrangement. Leaving OOK restores the FSK values the driver saved.
- It does not set deviation. After switching to an FSK mode, call `setDeviation()`. After switching to MSK, call `setMskPhaseTransition()`.
- It re-applies the PA table (section 6.6) for the new modulation.

For MSK below 500 kBd, TI recommends 2-FSK or GFSK with deviation equal to baud divided by four instead.

`bool setOokAgc(uint8_t magnTarget = 4, uint8_t agcCtrl0 = 0x92)`

Fine-tunes ASK/OOK receive, following TI note DN022. `magnTarget` is the whole AGCCTRL2 byte (0 to 7). `agcCtrl0` is the raw AGCCTRL0 byte: 0x91 gives an 8 dB decision boundary (the "low" bit must then be at least 16 dB below the "high" bit), and 0x92 gives 12 dB. AGCCTRL1 is set to 0x00. Returns `false` if you are not in ASK/OOK mode or `magnTarget` is above 7.

The optimum depends on data rate and bandwidth. DN022 says to find it by measuring packet error rate over the whole input range. The IF frequency (FSCTRL1) should also follow the RX bandwidth. That table is not in the note, so set it yourself with `writeReg(CC1101Reg::FSCTRL1, ...)` using SmartRF Studio values.

`bool setMskPhaseTransition(uint8_t m)`

MSK only. Sets DEVIATN.DEVIATION_M, the fraction of the symbol period used for the phase change. 0 is the SmartRF value for 500 kBd. Returns `false` if not in MSK mode or `m` is above 7.

`bool setChannelSpacing(float kHz)`

Sets the spacing between channel numbers. Accepted range is 25 to 406 kHz, otherwise `false`. The driver picks the closest achievable value. The power-on default the driver loads is about 200 kHz.

`void setChannel(uint8_t channel)`

Writes the channel number (CHANNR). The on-air frequency becomes the base frequency plus `channel` times the channel spacing. This lets you hop between channels cheaply without recomputing the frequency word. Make sure the result stays inside a supported band.

6.4 Packet handling

All of these drop the chip to IDLE. Call `startReceive()` afterwards.

`void setSyncWord(uint8_t hi, uint8_t lo)`

Sets the 16-bit sync word (SYNC1 = `hi`, SYNC0 = `lo`). The default is 0xD3 0x91. In the 32-bit sync modes the chip sends and expects the 16-bit word twice.

`bool setSyncMode(SyncMode mode)`

Selects how the sync word is sent and detected (table in 6.1). Returns `false` if `mode` is above 7. With `SYNC_NONE` and `SYNC_CS_ONLY` no preamble or sync word is sent in TX.

`bool setPreambleBytes(uint8_t n)`

Sets the number of preamble bytes sent in TX. The chip supports only 2, 3, 4, 6, 8, 12, 16 or 24, so the driver snaps your value to the nearest one (ties go to the lower value, so 5 becomes 4). Always returns `true`. Four bytes is the default and is recommended generally. Eight is recommended at 500 kBd.

`void setPreambleQuality(uint8_t pqt)`

Sets the preamble quality threshold (PQT), 0 to 7 (larger values are clamped to 7). The receiver only accepts a sync word after the preamble quality has reached the threshold. A value of 0 means a sync word is always accepted.

`void setCrc(bool enabled)`

Enables or disables the 16-bit CRC (on by default). Both ends must match. With CRC disabled, `readPacket()` never returns `ERR_CRC`.

`void setCrcAutoflush(bool enabled)`

When enabled, the chip flushes the RX FIFO itself when a packet fails its CRC. You then never see the bad packet and `ERR_CRC` will not be returned. The header notes that each packet must be read before the next arrives while this is on. Because `readPacket()` returns 0 without restarting reception when the FIFO is empty, check `marcState()` now and then and call `startReceive()` if the chip is not in `MARC_RX`, in case the chip stopped listening after discarding a packet.

`void setWhitening(bool enabled)`

Enables data whitening (off by default). Both ends must match.

`bool setManchester(bool enabled)`

Enables Manchester encoding and decoding. This halves the data rate for a given baud rate. Returns `false` (and changes nothing) when you try to enable it while in MSK or 4-FSK mode, or while FEC is on. Both ends must use the same setting. `manchester()` returns the current state.

Hardware address filtering

```
void setNodeAddress(uint8_t addr);
bool setAddressFilter(AddressFilter mode);
AddressFilter addressFilter() const;
uint8_t nodeAddress() const;
```

With filtering on, the first payload byte after the length byte is the destination address. The chip drops packets whose address does not match and restarts reception by itself. `setAddressFilter()` returns `false` for a mode above 3.

Practical rules:

- To send, use `transmitTo(dest, data, len)`, or put the destination address in `data[0]` yourself when using `transmit()`.
- On receive, `buf[0]` is the destination address and it is included in the length that `readPacket()` returns.
- Addresses 0x00 and 0xFF are the broadcast addresses recognised by the `ADDR_NODE_BC0` and `ADDR_NODE_BC0_BCFF` modes, so avoid using them as node addresses.
- Filtering cannot be used in infinite mode.

Example of a node that accepts its own address and broadcasts:

```
radio.setNodeAddress(0x42);
radio.setAddressFilter(CC1101Radio::ADDR_NODE_BC0);
radio.startReceive();
```

6.5 Packet length modes

These are for talking to devices with their own framing. The default is variable length.

`bool setVariableLength(uint8_t maxLen = MAX_PAYLOAD)`

A length byte is sent first, then the payload. `maxLen` (1 to 61) is the largest length byte the receiver will accept. Returns `false` for 0 or above 61. If FEC was on, it is turned off, because FEC needs fixed length.

`bool setFixedLength(uint8_t n)`

No length byte on air. The receiver accepts exactly `n` bytes, where `n` is 1 to 62 (the RX FIFO also holds two appended status bytes). Returns `false` for 0 or above 62.

On the transmit side, `transmit()` and `transmitTo()` may send **any** size from 1 to 64 per call in fixed mode. The packet-length register is switched for that one packet and put back afterwards, so your receive settings are unchanged. With address filtering, the address byte counts toward the size.

In fixed mode, the `maxLen` you give `readPacket()` must be at least the fixed length, or you get `ERR_BUF_SMALL`.

`bool setInfiniteLength()`

The radio keeps receiving after the sync word until you call `idle()`. Use `startReceive()` then `readStream()` in a loop. If FEC was on, it is turned off. In this mode:

- `transmit()`, `transmitTo()` and `readPacket()` return `ERR_BAD_ARG`. To send raw bytes, switch to fixed mode.
- CRC checking and address filtering do not apply to a stream.

`LengthMode lengthMode() const` and `uint8_t fixedLength() const`

Return the current mode and the configured fixed length.

`bool setFec(bool enabled)` and `bool fec() const`

Enables convolutional forward error correction with interleaving (datasheet section 17). It requires fixed-length mode and returns `false` otherwise. Enabling it turns Manchester off. It halves the data rate for a given baud rate. Switching to variable or infinite mode turns FEC off again. Both ends must match.

6.6 Output power

The nominal output power steps are -30, -20, -15, -10, 0, 5, 7 and 10 dBm. The matching PATABLE byte depends on the band (315, 433, 868 or 915 MHz) and, at 868/915 MHz, on the inductor type (section 8.3).

`int8_t setOutputPowerDbm(int8_t dbm)`

Snaps your request to the nearest step (ties go to the lower step; values above 10 become 10; values below -30 become -30), programs the PA and returns the dBm value actually chosen. `begin()` sets 0 dBm. Calling this clears any raw PA setting.

Unlike most setters, the output power functions do not force IDLE, so reception continues if the chip was receiving.

`void setPaRaw(uint8_t paValue)`

Writes a raw PATABLE byte. The datasheet says values 0x61 to 0x6F are not allowed (the driver does not enforce this). After this call, changing the frequency band or inductor type will not override your raw value. Calling `setOutputPowerDbm()` returns to table-driven power.

`void setInductorType(InductorType t)`

Chooses the multilayer (default) or wire-wound PATABLE set for 868/915 MHz boards. If you are using a dBm setting, the PA byte is refreshed immediately.

In FSK and MSK modes the power is held in PATABLE entry 0. In ASK/OOK mode the driver puts it in entry 1 and leaves entry 0 at zero (carrier off), and sets FREND0 accordingly.

6.7 State control

`void idle()`

Wakes the chip first if it was asleep, sends the IDLE command and waits up to 5 ms for the chip to report IDLE.

`void sleep()`

Goes to IDLE, saves the PATABLE contents and the test registers (FSTEST, TEST2, TEST1, TEST0), then sends the power-down command. The chip loses these registers in SLEEP and flushes both FIFOs, which is why the driver keeps copies. Sleep gives the lowest power consumption.

`void wake()`

Pulls CS low, waits up to 10 ms for the crystal to start, restores the saved registers and PATABLE, and waits for IDLE. The chip is left in IDLE, so call `startReceive()` to listen again. Calibration data survives SLEEP, so no recalibration is normally needed unless the temperature has changed a lot. Does nothing if the chip is not asleep.

Any driver function that calls `idle()` (all setters, `startReceive()`, `transmit()`) wakes the chip automatically. Avoid the raw access functions (6.11) while asleep, because they would physically wake the chip without the driver knowing. Call `wake()` first.

`void calibrate()`

Goes to IDLE, runs a manual synthesiser calibration (about 735 microseconds) and waits for IDLE. The chip already auto-calibrates on every IDLE to RX/TX transition, so you rarely need this.

`bool startReceive()`

Goes to IDLE, flushes the RX FIFO and enters RX. Always returns `true`. Any unread packet in the FIFO is discarded.

6.8 Transmitting

```
int16_t transmit(const uint8_t* data, uint8_t len, uint32_t timeoutMs = 0);
int16_t transmitTo(uint8_t dest, const uint8_t* data, uint8_t len, uint32_t
timeoutMs = 0);
```

`transmit()` sends `data`. `transmitTo()` sends `[dest][data...]` so receivers with address filtering pick it up. Both block until the packet is sent.

Limits on `len` (for `transmitTo()` the address byte counts too):

Mode	Allowed size per call
Variable length	1 to 61
Fixed length	1 to 64
Infinite	Not available (`ERR_BAD_ARG`)

What happens: the chip goes to IDLE, the TX FIFO is flushed, the packet (length byte if variable mode, optional address, payload) is loaded, and the transmit command is sent. The driver then polls the chip state until it returns to IDLE.

`timeoutMs` is the longest the driver waits. With the default of 0, the driver calculates it: the packet duration (preamble, sync, length, payload and CRC; doubled for Manchester or FEC; halved for 4-FSK) at the current baud rate, times 1.5, plus 20 ms.

Return values:

Value	Meaning
`ERR_NONE`	Packet sent. The chip is now in IDLE

Value	Meaning
`ERR_LENGTH`	`len` is 0 or too large for the mode
`ERR_BAD_ARG`	Infinite-length mode is active
`ERR_TX_UNDERFLOW`	The chip ran out of data mid-packet. The TX FIFO has been flushed
`ERR_TIMEOUT`	The chip did not return to IDLE in time. The driver forces IDLE and flushes the TX FIFO

Notes:

- The chip is in IDLE afterwards in every case. Call `startReceive()` to listen again.
- The driver does not listen before talking (the chip's clear-channel check is set to always clear). Observe your regulatory limits (duty cycle, power) yourself.
- The wait is a tight polling loop with no task yield, so a long packet at a low baud rate occupies the CPU for its whole duration. Keep this in mind for lower-priority tasks and watchdogs, or run the radio in its own task.

6.9 Receiving

`int16_t readPacket(uint8_t* buf, uint8_t maxLen, PacketInfo* info = nullptr)`

Checks for a complete packet, reads it and restarts reception. Call it regularly, for example every few milliseconds.

Return values:

Value	Meaning
Greater than 0	Payload length in bytes, copied to `buf` (includes the address byte when filtering is on)
0	No complete packet yet
`ERR_CRC`	Packet received but failed its CRC check. The payload is still copied to `buf` and `info` is filled, but treat the data as corrupt
`ERR_OVERFLOW`	The RX FIFO overflowed. Reception was restarted and data was lost

Value	Meaning
`ERR_LENGTH`	The length in the FIFO was invalid, or fewer bytes arrived than expected. Reception was restarted
`ERR_BUF_SMALL`	The packet is larger than `maxLen`. The packet is discarded and reception restarted
`ERR_BAD_ARG`	Infinite-length mode is active. Use `readStream()`

`buf` should be at least `MAX_PAYLOAD` (61) bytes to be safe in variable mode. `info` may be `nullptr`. It is only written when a packet is actually read (a positive result or `ERR_CRC`). The `rssi` and `lqi` values come from the status bytes the chip appended to the packet, and `broadcast` is set when the destination matched a broadcast address instead of your node address.

A packet is only read once the chip has left RX (that is, finished receiving it), so a packet in progress returns 0.

`int16_t readStream(uint8_t* buf, uint8_t maxLen)`

Infinite mode only (otherwise `ERR_BAD_ARG`). Copies up to `maxLen` bytes that have arrived so far and returns the count, or 0 if none. It always leaves one byte in the FIFO, as the datasheet requires while a packet is still being received. If the FIFO overflowed, reception is restarted and `ERR_OVERFLOW` is returned, meaning bytes were lost. Poll faster. The 64-byte FIFO fills in roughly `64 * 8 / baud` seconds (about 107 ms at 4.8 kBd, about 2 ms at 250 kBd). Call `idle()` to end the stream.

`int16_t available()`

Returns the number of bytes in the RX FIFO, or `ERR_OVERFLOW` if it has overflowed.

`float getRSSI()`

Returns the current signal strength in dBm. The chip should be in RX for a live reading. The value is converted with an offset of 74 dB, the datasheet figure for 433 and 868 MHz. At other bands treat the number as approximate.

`uint8_t packetStatus()` and `bool carrierSense()`

`packetStatus()` returns the raw PKTSTATUS register (read until two consecutive reads agree). `carrierSense()` returns the carrier sense flag (bit 6): true when the signal is above the carrier sense threshold.

`float frequencyErrorKHz()`

Returns the chip's frequency offset estimate in kHz (each step is about 1.59 kHz). Meaningful for FSK and MSK reception only. Useful for checking how far the other end's crystal is from yours.

6.10 Chip information

- `uint8_t partNumber()`: the PARTNUM status register (0x00 for the CC1101).
- `uint8_t version()`: the VERSION register. Genuine parts report 0x04 or 0x14.
- `uint8_t marcState()`: the main radio control state (section 8.5), read until two reads agree. Useful for diagnostics, for example confirming the chip is in `MARC_RX`.

6.11 Raw register access

Use these for anything the helpers do not cover. The register and command names are in the `CC1101Reg` namespace (section 8.5).

Function	Purpose
`uint8_t readReg(uint8_t reg)`	Reads one configuration register (or one FIFO byte)
`void writeReg(uint8_t reg, uint8_t value)`	Writes one configuration register
`uint8_t readStatus(uint8_t reg)`	Reads a status register (addresses 0x30 to 0x3D). Status registers share addresses with command strobes, so this sets the burst bit
`uint8_t strobe(uint8_t cmd)`	Sends a command strobe such as `SIDLE` or `SRX`, and returns the chip status byte
`void readBurst(uint8_t reg, uint8_t* buf, uint8_t len)`	Reads `len` bytes starting at `reg` (at most 64)
`void writeBurst(uint8_t reg, const uint8_t* buf, uint8_t len)`	Writes `len` bytes starting at `reg` (at most 64). Writing to `PATABLE` loads the power table

Example, reading the chip state and setting a register not covered by the helpers:

```
using namespace CC1101Reg;
uint8_t state = radio.readStatus(MARCSTATE) & 0x1F;
radio.idle();
radio.writeReg(FSCTRL1, 0x08);
```

Cautions:

- Registers written directly are not tracked by the driver. If you change registers the driver also manages (for example MDMCFG2 or PKTCTRL0), its internal state can disagree with the chip.
- Strobes sent directly bypass the driver's sleep tracking. Use `sleep()` and `wake()` rather than `strobe(SPWD)`.
- Most registers should only be changed in IDLE. Call `idle()` first.

7. Recipes

7.1 Two radios on one SPI bus

Each radio needs its own CS pin. All radios share SCK, MOSI and MISO, and every radio gets the same `misoPin`. Initialise the bus once.

```
static CC1101Radio radioA(SPI2_HOST, 5, 19);   // CS 5, MISO 19
static CC1101Radio radioB(SPI2_HOST, 4, 19);   // CS 4, MISO 19

radioA.begin(433.92f);
radioB.begin(868.30f);
radioA.startReceive();
radioB.startReceive();
```

In the polling loop, call `readPacket()` on each in turn. Remember that each object should be used from one task only (section 4.7).

7.2 Addressed network

A node with address 0x42 that accepts its own address and broadcasts, and sends to node 0x10:

```
radio.begin();
radio.setNodeAddress(0x42);
radio.setAddressFilter(CC1101Radio::ADDR_NODE_BC0);
radio.startReceive();

const uint8_t msg[] = {1, 2, 3};
radio.transmitTo(0x10, msg, sizeof msg);
radio.startReceive();

// receiving
uint8_t buf[CC1101Radio::MAX_PAYLOAD];
CC1101Radio::PacketInfo info;
int16_t n = radio.readPacket(buf, sizeof buf, &info);
if (n > 0) {
  // buf[0] is the destination address, buf[1..n-1] is the payload
  // info.broadcast tells you whether it was a broadcast
}
```

7.3 ASK/OOK receive

```
radio.begin(433.92f, 3.8f, 0.0f, 100.0f, CC1101Radio::MOD_ASK_OOK);
radio.setOokAgc(4, 0x92);   // optional: tune per TI DN022
radio.startReceive();
```

The deviation argument is ignored for ASK/OOK. The 100 kHz request snaps to the nearest chip bandwidth (101.6 kHz). `setOokAgc()` is only needed if you want to tune beyond the defaults `begin()` applies. The driver only provides packet-based OOK. It has no helper for the chip's asynchronous serial mode.

7.4 Continuous byte stream (infinite length)

```
radio.setInfiniteLength();
radio.startReceive();
uint8_t chunk[32];
for (;;) {
  int16_t n = radio.readStream(chunk, sizeof chunk);
  if (n > 0) { /* use chunk[0..n-1] */ }
  else if (n == CC1101Radio::ERR_OVERFLOW) { /* bytes were lost: poll faster
*/ }
  // delay shorter than the FIFO fill time (section 6.9)
}
```

Note that a FreeRTOS tick delay of one tick can be as long as 10 ms (at the default 100 Hz tick), which is too slow at high baud rates. At 4.8 kBd the 64-byte FIFO takes about 107 ms to fill, so a short delay is fine.

7.5 Fixed-length packets with FEC

```
radio.setFixedLength(16);
radio.setFec(true);        // also turns Manchester off
radio.startReceive();
```

Both ends must use the same fixed length and FEC setting. The data rate is half the baud rate. On the transmit side you may still send other sizes per call in fixed mode (1 to 64 bytes), but a receiver set to 16 bytes will only accept 16.

7.6 Low-power sleep

```
radio.sleep();      // lowest power; the driver saves the registers SLEEP
loses
// ... do other work, deep sleep the MCU, etc. ...
radio.wake();       // chip returns to IDLE
radio.startReceive();
```

7.7 Channel hopping

```
radio.setChannelSpacing(200.0f);
for (uint8_t ch = 0; ch < 8; ch++) {
  radio.setChannel(ch);
  radio.startReceive();
  // ... listen for a while ...
}
```

Both ends must hop in step. The on-air frequency for each step is the base frequency plus the channel times the spacing.

7.8 Measuring link quality

For each packet, `PacketInfo` gives `rssi` and `lqi`. For a live reading of the channel while receiving, use `getRSSI()` and `carrierSense()`. To check whether the two ends' crystals agree, read `frequencyErrorKHz()` after receiving.

8. Reference tables

8.1 Return codes

Constant	Value	Meaning
`ERR_NONE`	0	Success
`ERR_CHIP_NOT_FOUND`	-1	`begin()` read VERSION as 0x00 or 0xFF
`ERR_BAD_ARG`	-2	Argument or settings combination rejected, or the call is not valid in the current length mode
`ERR_TIMEOUT`	-3	`transmit()` did not finish in time
`ERR_OVERFLOW`	-4	RX FIFO overflowed (data lost)
`ERR_CRC`	-5	Packet failed its CRC check
`ERR_BUF_SMALL`	-6	Your buffer (`maxLen`) is smaller than the packet
`ERR_LENGTH`	-7	Invalid payload length (TX) or invalid or incomplete length in the FIFO (RX)
`ERR_TX_UNDERFLOW`	-8	TX FIFO ran empty during transmission
`ERR_SPI`	-9	The device could not be added to the SPI bus

`readPacket()` returns the payload length when it is zero or more, so positive values are never error codes.

8.2 Register defaults written by `begin()`

These are applied after the chip is reset (based on SmartRF Studio output for 433 MHz, GFSK, 4.8 kBd). Frequency, baud rate, deviation, bandwidth, modulation and power are then applied on top.

Register	Value	Notes
IOCFG0	0x06	GDO0 asserts on sync word, deasserts at end of packet
FIFOTHR	0x47	ADC retention on, FIFO threshold 7

Register	Value	Notes
SYNC1, SYNC0	0xD3, 0x91	Default sync word
PKTLEN	0x3D	Maximum packet length 61
PKTCTRL1	0x04	Append RSSI/LQI/CRC-OK status bytes, no address check
PKTCTRL0	0x05	Variable length, CRC on, no whitening
ADDR	0x00	Node address
FSCTRL1, FSCTRL0	0x06, 0x00	IF frequency, frequency offset
MDMCFG2	0x02	16/16 sync, no Manchester (modulation field set afterwards)
MDMCFG1	0x22	No FEC, 4 preamble bytes, channel spacing exponent 2
MDMCFG0	0xF8	Channel spacing mantissa (about 200 kHz together with MDMCFG1)
MCSM1	0x00	Always clear-to-send, IDLE after RX and after TX
MCSM0	0x18	Auto-calibrate on IDLE to RX/TX
FOCCFG	0x16	Frequency offset compensation
BSCFG	0x6C	Bit synchronisation
AGCCTRL2, 1, 0	0x43, 0x40, 0x91	AGC settings (FSK)
WORCTRL	0xFB	Wake-on-radio control (no helper functions use it)
FREND1	0x56	Front-end RX
FSCAL3, 2, 1, 0	0xE9, 0x2A, 0x00, 0x1F	Frequency synthesiser calibration
FSTEST	0x59	
TEST2, TEST1, TEST0	0x81, 0x35, 0x09	Per SmartRF Studio

8.3 Output power tables (PATABLE bytes)

The driver picks the column that matches the base frequency: below 360 MHz is 315, below 779 MHz is 433, below 900 MHz is 868, otherwise 915. The 868 and 915 columns come in multilayer (ML, default) and wire-wound (WW) variants.

dBm	315	433	868 ML	915 ML	868 WW	915 WW
-30	0x12	0x12	0x03	0x03	0x03	0x03
-20	0x0D	0x0E	0x0F	0x0E	0x17	0x0E
-15	0x1C	0x1D	0x1E	0x1E	0x1D	0x1E
-10	0x34	0x34	0x27	0x27	0x26	0x27
0	0x51	0x60	0x50	0x8E	0x50	0x8E
5	0x85	0x84	0x81	0xCD	0x86	0x84
7	0xCB	0xC8	0xCB	0xC7	0xCD	0xCC
10	0xC2	0xC0	0xC2	0xC0	0xC5	0xC3

These are the datasheet's nominal values (Tables 37 and 39). Actual output depends on your board.

8.4 Receiver bandwidths available

The chip offers these 16 values (kHz). `setRxBandwidth()` picks the closest one to your request.

Exponent	Mantissa 0	Mantissa 1	Mantissa 2	Mantissa 3
0	812.5	650.0	541.7	464.3
1	406.3	325.0	270.8	232.1
2	203.1	162.5	135.4	116.1
3	101.6	81.3	67.7	58.0

For example, the default request of 135 kHz becomes 135.4 kHz.

8.5 `CC1101Reg` namespace

The header defines the whole register map, so you can use the names with the raw access functions (`using namespace CC1101Reg;`).

Configuration registers (0x00 to 0x2E, plus PATABLE and FIFO):

Registers	Addresses
IOCFG2, IOCFG1, IOCFG0, FIFOTHR	0x00 to 0x03
SYNC1, SYNC0, PKTLEN, PKTCTRL1, PKTCTRL0, ADDR, CHANNR	0x04 to 0x0A
FSCTRL1, FSCTRL0	0x0B, 0x0C
FREQ2, FREQ1, FREQ0	0x0D to 0x0F
MDMCFG4, MDMCFG3, MDMCFG2, MDMCFG1, MDMCFG0	0x10 to 0x14
DEVIATN, MCSM2, MCSM1, MCSM0	0x15 to 0x18
FOCCFG, BSCFG, AGCCTRL2, AGCCTRL1, AGCCTRL0	0x19 to 0x1D
WOREVT1, WOREVT0, WORCTRL	0x1E to 0x20
FREND1, FREND0	0x21, 0x22
FSCAL3, FSCAL2, FSCAL1, FSCAL0	0x23 to 0x26
RCCTRL1, RCCTRL0, FSTEST, PTEST, AGCTEST	0x27 to 0x2B
TEST2, TEST1, TEST0	0x2C to 0x2E
PATABLE, FIFO	0x3E, 0x3F

Command strobes:

Name	Address	Action
SRES	0x30	Reset chip
SFSTXON	0x31	Enable and calibrate the frequency synthesiser
SXOFF	0x32	Turn off the crystal oscillator
SCAL	0x33	Calibrate the frequency synthesiser
SRX	0x34	Enable RX

Name	Address	Action
STX	0x35	Enable TX
SIDLE	0x36	Go to IDLE
SWOR	0x38	Start wake-on-radio
SPWD	0x39	Enter power-down when CS goes high
SFRX	0x3A	Flush RX FIFO
SFTX	0x3B	Flush TX FIFO
SWORRST	0x3C	Reset the wake-on-radio timer
SNOP	0x3D	No operation

Status registers (read with `readStatus()`):

Name	Address
PARTNUM	0x30
VERSION	0x31
FREQEST	0x32
LQI	0x33
RSSI	0x34
MARCSTATE	0x35
WORTIME1, WORTIME0	0x36, 0x37
PKTSTATUS	0x38
VCO_VC_DAC	0x39
TXBYTES	0x3A
RXBYTES	0x3B
RCCTRL1_STATUS, RCCTRL0_STATUS	0x3C, 0x3D

`MARCSTATE` values (datasheet 29.3), as returned by `marcState()`:

Constant	Value	State
`MARC_SLEEP`	0x00	Sleep
`MARC_IDLE`	0x01	Idle
`MARC_RX`	0x0D	Receiving
`MARC_RX_END`	0x0E	End of reception
`MARC_RX_RST`	0x0F	RX restarting
`MARC_RX_OVERFLOW`	0x11	RX FIFO overflow
`MARC_FSTXON`	0x12	Synthesiser on, ready for TX
`MARC_TX`	0x13	Transmitting
`MARC_TX_END`	0x14	End of transmission
`MARC_TX_UNDERFLOW`	0x16	TX FIFO underflow

8.6 Size limits by mode

Mode	Largest packet to send	Largest packet to receive
Variable length	61 bytes	61 bytes (set lower with `setVariableLength()`)
Fixed length	64 bytes per call	The configured length, at most 62 bytes
Infinite	Not available	Unlimited stream via `readStream()`

With address filtering the address byte counts toward these sizes.

9. Troubleshooting

Symptom	Likely cause and what to try
`begin()` returns `ERR_SPI` (-9)	`spi_bus_initialize()` was not called first, was called for a different host, or the bus has no free device slots. Check the host passed to the constructor matches the bus

Symptom	Likely cause and what to try
`begin()` returns `ERR_CHIP_NOT_FOUND` (-1)	VERSION read as 0x00 or 0xFF. Check power (3.3 V), ground, all four SPI wires, the CS pin number and that MISO is on the pin you passed. A floating MISO often reads 0xFF, and MISO shorted to ground reads 0x00
`begin()` returns `ERR_BAD_ARG` (-2)	A setting is out of range: frequency outside the supported bands, baud rate outside the modulation's limits, deviation outside 1.5 to 381 kHz, or bandwidth outside 58 to 812 kHz. The chip is left partly configured, so fix the value and call `begin()` again
Nothing is received	Check that both ends match on frequency, modulation, baud rate, deviation, sync word and mode, length mode, CRC, whitening, Manchester and FEC (section 4.5). Make sure you called `startReceive()` and are calling `readPacket()` often enough
Radio goes deaf after sending	`transmit()` leaves the chip in IDLE. Call `startReceive()` after every transmit
Packets are missed when traffic is fast	After each packet the chip waits in IDLE until `readPacket()` restarts reception. Poll more often, and keep the time between a packet's arrival and your next poll short
Frequent `ERR_CRC`	Weak signal, frequency error, interference, or mismatched CRC or whitening settings. Compare `info.rssi` and `frequencyErrorKHz()`. Try a wider RX bandwidth if the two crystals disagree
`ERR_LENGTH` on receive	The length byte was invalid or the packet was incomplete. Often caused by mismatched length mode, whitening or Manchester settings, or noise triggering a false sync detection. Try a longer sync word mode (`SYNC_30_32`) or a higher preamble quality threshold

Symptom	Likely cause and what to try
`ERR_BUF_SMALL`	Your `maxLen` is smaller than the packet. Use a `MAX_PAYLOAD`-sized buffer. In fixed mode `maxLen` must be at least the fixed length
`ERR_OVERFLOW`	Packets or stream bytes arrived faster than you read them. Poll faster
`transmit()` returns `ERR_TIMEOUT`	The chip never finished sending. Check the supply voltage and stability, and consider passing a larger `timeoutMs` for very slow baud rates
`transmit()` returns `ERR_LENGTH`	`len` is 0, or larger than 61 in variable mode (64 in fixed mode). Remember `transmitTo()` adds one byte
`transmit()` returns `ERR_BAD_ARG`	Infinite-length mode is active. Switch to fixed or variable mode
`setManchester(true)` returns `false`	MSK, 4-FSK or FEC is active. Manchester cannot be combined with them
`setFec(true)` returns `false`	FEC needs fixed-length mode. Call `setFixedLength()` first
Range is poor	Check antenna and matching for your band, the output power setting, and the inductor type on 868/915 MHz boards. Lower baud rates and narrower bandwidths improve sensitivity
Wrong frequency or baud rate on air	The module probably does not have a 26 MHz crystal. The driver assumes 26 MHz
Settings seem lost after `reset()`	`reset()` restores chip defaults without reapplying your configuration. Call `begin()` again
Watchdog warnings during long transmits	`transmit()` busy-waits without yielding. Run the radio in its own task, and consider the packet duration at your baud rate

10. Behaviours and limitations to keep in mind

- **Polled operation.** The driver uses no interrupts and does not use the GDO pins. You must call `readPacket()` or `readStream()` regularly.
- **IDLE after every packet.** Because of the chip configuration (MCSM1 = 0x00), reception stops after each packet until `readPacket()` or `startReceive()` restarts it, and transmission always ends in IDLE.
- **Blocking transmit.** `transmit()` and `transmitTo()` busy-wait until the packet is sent, without yielding to other tasks.
- **No listen-before-talk.** The chip's clear-channel assessment is set to always clear. Add your own check (for example `carrierSense()`) if your regulations require it.
- **26 MHz crystal assumed.** All derived values depend on it.
- **Not thread-safe per object.** Use each radio object from one task or add your own locking.
- **Setters mostly require re-arming.** Configuration functions drop the chip to IDLE. Always call `startReceive()` afterwards. The output power functions are the exception.
- **`setDeviation()` is not checked against the modulation.** Use it only for 2-FSK, GFSK and 4-FSK.
- **`setModulation()` does not re-validate the baud rate or set the deviation.** Do both yourself after switching.
- **`reset()` does not reconfigure.** Use `begin()` to start over.
- **Bad-CRC packets still return data.** `ERR_CRC` comes with the corrupt payload in `buf` and a filled `PacketInfo`. Ignore the bytes unless you have a reason to inspect them.
- **RSSI offset is a single constant.** It is the 433/868 MHz figure, so RSSI at other bands is approximate.
- **Raw access bypasses the driver's bookkeeping.** Direct register writes, and strobes such as `SPWD`, are not tracked.
- **Wake-on-radio, asynchronous and synchronous serial modes, and GDO signal routing** have no helper functions. They can be set up with the raw register access functions if you need them.
- **SPI setup.** The example is built around DMA disabled and a 64-byte maximum transfer size on the bus.